



UNITED INTERNATIONAL UNIVERSITY

Department of Computer Science and Engineering

Artificial Intelligence Laboratory

Course No: CSE 3812

Intelligent Sudoku Solver

Using CSP, AC-3, Backtracking and MRV

Submitted By : MD. Mishkatul Bari Sifat

Student ID : 0112330669

Section : H

Trimester : Summer 2026 (Online-2)

Submitted To : Kazi Fahim Tahmid (KFT)

Designation : Lecturer

Email : fahimtahmid@cse.uiu.ac.bd

United International University
Dhaka, Bangladesh

Contents

1	Introduction	2
2	Problem Representation as CSP	2
2.1	Variables	2
2.2	Domains	2
2.3	Constraints	2
3	Input and Output Specification	3
4	Algorithm Design	3
4.1	AC-3 Algorithm	3
4.2	Backtracking Search	3
4.3	Minimum Remaining Values (MRV) Heuristic	4
5	Object-Oriented Implementation	4
6	Experimental Results and Ablation Study	4
6.1	MRV Enabled Results	5
6.2	MRV Disabled Results	5
7	Discussion	5
8	Conclusion	5
9	Future Improvements	6

1 Introduction

Sudoku is a classic constraint-based puzzle that can be modeled as a Constraint Satisfaction Problem (CSP).

The objective of this project is to develop an Artificial Intelligence based Sudoku solver that efficiently solves Sudoku puzzles using intelligent search and constraint propagation techniques.

Instead of using a naive brute-force approach, this project implements CSP representation, AC-3 (Arc Consistency-3), Backtracking Search, and Minimum Remaining Values (MRV) heuristic.

The main goal is to reduce unnecessary search operations and improve solving efficiency.

2 Problem Representation as CSP

A Constraint Satisfaction Problem consists of three main components:

2.1 Variables

The Sudoku board contains 81 variables where each cell represents one variable.

$$Total\ Variables = 9 \times 9 = 81$$

2.2 Domains

The domain represents possible values for each variable.

For an empty cell:

$$Domain = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

For a pre-filled cell:

$$Domain = \{Given\ Value\}$$

2.3 Constraints

Sudoku contains three main constraints:

- No duplicate number can exist in the same row.
- No duplicate number can exist in the same column.
- No duplicate number can exist inside the same 3x3 sub-grid.

3 Input and Output Specification

The program reads Sudoku puzzles from:

`puzzles.txt`

Each puzzle is represented as an 81-character string.

The first 9 characters represent the first row, the next 9 characters represent the second row, and so on.

The digit 0 represents an empty cell.

After solving, the program generates:

`solutions.txt`

The output file contains solved Sudoku boards represented as 81-character strings without zero values.

4 Algorithm Design

4.1 AC-3 Algorithm

AC-3 (Arc Consistency-3) is used as a preprocessing algorithm before applying search.

The purpose of AC-3 is to reduce variable domains by removing values that cannot satisfy Sudoku constraints.

The algorithm checks every pair of connected variables called arcs:

$$(X_i, X_j)$$

If a value in one variable has no supporting value in another variable, that value is removed from the domain.

Advantages of AC-3:

- Reduces search space.
- Removes invalid values early.
- Improves backtracking performance.

4.2 Backtracking Search

After AC-3 preprocessing, recursive Backtracking Search is used to find the solution.

The process:

1. Select an unassigned variable.
2. Try a possible value.
3. Check constraint satisfaction.
4. Continue recursively.
5. Backtrack if contradiction occurs.

4.3 Minimum Remaining Values (MRV) Heuristic

MRV is a variable selection heuristic.

It selects the variable with the smallest remaining domain.

Example:

$$CellA = \{1, 2, 3, 4\}$$

$$CellB = \{7\}$$

MRV selects Cell B first.

Benefits of MRV:

- Reduces unnecessary branching.
- Detects failure earlier.
- Decreases node expansion.

5 Object-Oriented Implementation

The solver was implemented using an object-oriented approach.

The main class is:

SudokuCSP

The class manages:

- Variables and domains
- Neighbor generation
- Constraint checking
- AC-3 execution
- Backtracking search
- Solution generation

This structure improves code organization and maintainability.

6 Experimental Results and Ablation Study

To evaluate the effect of the MRV heuristic, two experiments were performed:

1. MRV Enabled
2. MRV Disabled

6.1 MRV Enabled Results

The solver was executed with MRV heuristic enabled.

Puzzle	Execution Time	Node Expansion
Puzzle 1	0.00548 sec	1
Puzzle 2	8.4245 sec	6337
Puzzle 3	1.3556 sec	950

Table 1: Performance with MRV Enabled

6.2 MRV Disabled Results

The solver was executed without MRV heuristic using sequential variable selection.

Puzzle	Execution Time	Node Expansion
Puzzle 1	0.00640 sec	1
Puzzle 2	553.1327 sec	347581
Puzzle 3	0.2813 sec	177

Table 2: Performance with MRV Disabled

7 Discussion

The experimental results demonstrate the effectiveness of the MRV heuristic.

For the second puzzle:

Method	Time	Node Expansion
MRV Enabled	8.4245 sec	6337
MRV Disabled	553.1327 sec	347581

Table 3: MRV Performance Comparison

Without MRV, the solver explores many unnecessary branches because variables are selected sequentially.

With MRV, the most constrained variable is selected first, which reduces the search space and improves efficiency.

8 Conclusion

An intelligent Sudoku solver was successfully developed using Artificial Intelligence techniques.

The Sudoku puzzle was modeled as a Constraint Satisfaction Problem and solved using:

- CSP representation

- AC-3 constraint propagation
- Backtracking Search
- MRV heuristic

The experimental results show that MRV significantly improves performance by reducing unnecessary node expansion and execution time.

9 Future Improvements

Possible future improvements include:

- Degree heuristic implementation.
- Least Constraining Value ordering.
- Visualization of solving process.
- Support for larger Sudoku variants.

References

1. Artificial Intelligence Laboratory Course Materials.